

RoboPet - Combat Balance and Rooms Polish

Cursor task - paste this in and run it

Three changes: make combat winnable and add progression, put the dog in the other rooms, and make the dog stand still in the Combat room.

1. Combat (Part A): add a level-scaled enemy so Level 1 is winnable, plus win / lose handling, a scrap reward, and level progression.
2. Rooms (Part B): in the Fabricator and Laboratory rooms, remove the placeholder box and add the RobotDog (wandering).
3. Dog (Part C): add an autoWander flag to RobotDog. In the Combat room, add the dog with autoWander: false so it stands still like a statue.
4. Run flutter pub get, hot restart, then verify every item in the Acceptance Test.

Acceptance test

- [] Level 1 combat is winnable with default stats (the player can beat the first enemy).
- [] Winning grants scrap (top-left counter goes up) and unlocks the next level.
- [] Losing shows a defeat state and lets you retry at full HP without losing your level.
- [] Higher levels have tougher enemies (HP / ATK / DEF scale up).
- [] Fabricator and Laboratory show the dog instead of the placeholder box.
- [] In Fabricator and Laboratory the dog wanders normally.
- [] In the Combat room the dog stands still (no wandering), like a statue.

Part A - Combat balance and progression

Why Level 1 is currently unbeatable

With the current stats (player ATK 15 / DEF 10 / regen 2, enemy HP 120 / ATK 18 / DEF 12) the enemy deals $18 - 10 \times 0.3 = 15$ per tick, minus 2 regen = 13 net, so the player (HP 100) dies in about 8 ticks. The player deals $15 - 12 \times 0.3 = 11$ per tick and needs about 11 ticks to drop the enemy. Because both sides tick at the same speed, the player always dies first - Level 1 is mathematically unwinnable. The fix below makes Level 1 enemies weak and scales difficulty up per level.

Level-scaled enemy

```
class EnemyConfig {
    final int hp;
    final int atk;
    final int def;
    final int spd;
    const EnemyConfig({
        required this.hp,
        required this.atk,
```

```

        required this.def,
        required this.spd,
    });
}

// Level 1 is intentionally easy so new players can win and learn the loop.
EnemyConfig enemyForLevel(int level) {
    final l = (level - 1).clamp(0, 999);
    return EnemyConfig(
        hp: 55 + l * 25,
        atk: 9 + l * 3,
        def: 5 + l * 2,
        spd: 9 + l,
    );
}

```

Level 1 enemy becomes HP 55 / ATK 9 / DEF 5. The player now deals about 14 per tick (kills in ~4 ticks) and only takes ~4 net per tick, so Level 1 is a comfortable win.

Start a fight from the current level

```

void startFight() {
    // state.level is the current unlocked level and starts at 1.
    final cfg = enemyForLevel(state.level);
    state = state.copyWith(
        enemyMaxHp: cfg.hp,
        enemyHp: cfg.hp,
        enemyAtk: cfg.atk,
        enemyDef: cfg.def,
        enemySpd: cfg.spd,
        playerHp: state.playerMaxHp,
        phase: CombatPhase.fighting,
    );
    _timer = Timer.periodic(const Duration(milliseconds: 1600), (_) => _tick());
}

```

Resolve each tick, then win or lose

```

void _tick() {
    // Player hits enemy.
    final toEnemy = (state.playerAtk - state.enemyDef * 0.3).round().clamp(1, 9999);
    final enemyHp = (state.enemyHp - toEnemy).clamp(0, state.enemyMaxHp);

    // Enemy hits player, then the player regenerates a little.
    final toPlayer = (state.enemyAtk - state.playerDef * 0.3).round().clamp(1, 9999);
    final playerHp =
        (state.playerHp - toPlayer + state.playerRegen).clamp(0, state.playerMaxHp);

    state = state.copyWith(enemyHp: enemyHp, playerHp: playerHp);

    if (enemyHp <= 0) {
        _onVictory();
    } else if (playerHp <= 0) {
        _onDefeat();
    }
}

void _onVictory() {
    _timer?.cancel();
    final reward = 10 + state.level * 5;
    ref.read(economyProvider.notifier).addScrap(reward);
    state = state.copyWith(

```

```

        level: state.level + 1, // unlock next level
        phase: CombatPhase.victory,
        lastReward: reward,
    );
}

void _onDefeat() {
    _timer?.cancel();
    state = state.copyWith(phase: CombatPhase.defeat);
}

void retry() {
    // Same level, full HP, fight again.
    startFight();
}

```

Optional - active attack

For more engagement, add an Attack button that sets a one-shot multiplier on the next player hit (for example 1.8x) with a short cooldown. Apply it inside `_tick` before clamping to `Enemy`, then reset it. Keep it optional - the balance fix above already makes Level 1 winnable.

Part B - Dog in Fabricator and Laboratory

The other rooms still show a placeholder box. Swap it for the dog so the pet is present everywhere.

```

// In FabricatorWorld.onLoad() and LaboratoryWorld.onLoad():
// 1) DELETE the placeholder box component (the colored square with its label).
// 2) ADD the dog so it lives in these rooms too (wandering is fine here).
add(RobotDog(Vector2(0, 250)));

```

Part C - Dog stands still in the Combat room

Add an `autoWander` flag to `RobotDog`. When it is false, the dog never picks wander targets and never moves - it just stands there.

```

class RobotDog extends SpriteAnimationGroupComponent<DogAnim> with TapCallbacks {
    final bool autoWander;

    RobotDog(Vector2 position, {this.autoWander = true})
        : super(
            position: position,
            size: Vector2.all(160),
            anchor: Anchor.bottomCenter,
            priority: 10,
        );

    @override
    Future<void> onLoad() async {
        // ... keep your existing animation setup ...
        current = DogAnim.idle;
        startWander(); // self-guards on autoWander below
    }

    void startWander() {
        if (!autoWander) return; // Combat dog stands still like a statue.
        // ... keep your existing wander start logic ...
    }
}

```

```

@Override
void update(double dt) {
    super.update(dt);
    if (!autoWander) return; // No movement in the Combat room.
    // ... keep your existing wander / movement logic ...
}
}

```

Then place the dog in the Combat room with wandering off:

```

// In CombatWorld.onLoad(): add the dog as the fighter on the left, standing still.
add(RobotDog(Vector2(-120, 250), autoWander: false));

```

Notes

- Adapt names: `state.playerAtk`, `state.playerRegen`, `economyProvider.addScrap`, `CombatPhase`, and the world class names may differ in your code. The logic is what matters, not the exact identifiers.
- Reward tuning: scrap reward is $10 + \text{level} \times 5$ (Level 1 gives 15). This is what makes the top-left scrap counter start moving; adjust to taste.
- Defeat keeps your level: `retry()` restores full HP and refights the same level, so a loss is never a hard wall.
- Combat dog position: `Vector2(-120, 250)` puts it on the left. Place the enemy on the right so they face each other.